TAF course (*Preliminary*)

Development of a pythonic Template of Analysis Framework (TAF)

◇ Git & Github project creation ◇ PYTHON tips & tricks ◇
◇ Analysis workflow development ◇
◇ Worlflow configuration ◇

TAF course (*Preliminary*)- October 8, 2026

General notations of the document

Definition

Rule or convention

Method

Ex. 0.1 | Exercise

Nota bene

Remark

Achtung!

Bonus

A few disclaimers :

- The pieces of advice given in this course are not universal nor unique, as there are often many possible approaches to a problem.
- This course is adapted to Unix and Mac operating systems. On Windows computer, one would have to work with a Unix kernel (as WSL).
- This is not a PYTHON course per say but several PYTHON-dedicated tips & tricks are scattered along the course.

Prerequisites

- Programming :
  - some basic git knowledge
    > 💬 Not mandatory but preferable as the git functionalities used will not be presented in detail.
  - some basic PYTHON knowledge
    > 💬 PYTHON console, execution of a PYTHON script, variables, functions, ...
  - some basic Bash knowledge
    > 💬 Command lines in terminal, environment variables, ...
- Tools that must already be installed/prepared :
  - git
    > 💬 Should be natively available on all machines.
  - a github account
    > 💬 Ideally with SSH key configured, but not mandatory.
  - Visual Studio Code
    > 💬 Not mandatory but preferable as there will be dedicated exercises.
  - miniconda
    > 💬 Not mandatory but preferable as there will be dedicated exercises.

> ⚠ We will not be able do perform some "*installation customer service*" during this course given the time constraints.

TAF course (*Preliminary*)- October 8, 2026

Let's assume the following scenario :

You are a PhD student that just started the PhD. Your supervisor told you that you are going to pursue the work of a former PhD student, and that this student left his/her codes in a folder on a disc. Great! You think that you are lucky and that your analysis will quickly be on track... until you open the folder left by the former student : the files are not organised, the codes are not documented, there are no indications on how to use them, and the path to the files are hardcoded and are the absolute paths only valid in the machine of the former PhD student. And, even after struggling to understand how to run one of those codes, you realise that you are lacking some libraries for the codes to run!



The goal of this course is to give you the tools to build an Analysis Framework with files organisation and versioning, indications on how to read and run the codes, configurable workflows, ... in a few words : the complete opposite of the situation described above. This way, your colleagues or the next PhD student will not face such struggles.

Course layout

Chapter 1 - Creation of the project **5**

I) Set up version control . . . 5
- I.1) Creation of a github repository . . . 5
- I.2) Local creation of a git project . . . 6
- I.3) Create a `dev` branch . . . 6
- I.4) Open a Pull Request . . . 8

II) Set up a virtual environment . . . 9
- II.1) Motivations . . . 9
- II.2) Using `conda` . . . 10
  - II.2.a) Write a configuration file . . . 10
  - II.2.b) The environment creation in itself . . . 11
- II.3) Using PYTHON `venv` . . . 12
  - II.3.a) Write a configuration file . . . 12
  - II.3.b) The environment creation in itself . . . 12

III) Installation guide in `README.md` . . . 13
- III.1) For the virtual environment creation . . . 13
- III.2) Create an environment variable for the main repository . . . 14
  - III.2.a) Creation and use . . . 14
  - III.2.b) Update the installation guide . . . 16

Chapter 2 - PYTHON tips & tricks **17**

I) Set up Visual Studio Code for PYTHON . . . 18

II) Some PYTHON tips for aesthetics (but not only…) . . . 19
- II.1) The `if __name__ == "__main__"` . . . 19
- II.2) The C-like architecture . . . 22
- II.3) Type hint . . . 23
- II.4) Comments & docstrings . . . 24
  - II.4.a) Docstring for a function or a method . . . 24
  - II.4.b) Docstring for a whole file . . . 25
  - II.4.c) Highlighted comments . . . 25
- II.5) Add flags to the `main` function . . . 26
- II.6) LOG messages . . . 27
  - II.6.a) The `Utils` folder . . . 27
  - II.6.b) Create a logger module . . . 27
  - II.6.c) Update `PYTHONPATH` . . . 27
  - II.6.d) Update installation guide in `README.md` . . . 29
- II.7) Create a Visual Studio Code snippet . . . 29

III) Some PYTHON tips for the actual development . . . 31
- III.1) `f-string` . . . 31

TAF course (*Preliminary*)- October 8, 2026

III.2) `list` 31
III.3) `dict` 33
III.4) Loop over iterables 34

TAF course (*Preliminary*)- October 8, 2026

Chapter 1

Creation of the project

Chapter outline

I) Set up version control . . . **5**
- I.1) Creation of a github repository . . . 5
- I.2) Local creation of a git project . . . 6
- I.3) Create a `dev` branch . . . 6
- I.4) Open a Pull Request . . . 8

II) Set up a virtual environment . . . **9**
- II.1) Motivations . . . 9
- II.2) Using `conda` . . . 10
  - II.2.a) Write a configuration file . . . 10
  - II.2.b) The environment creation in itself . . . 11
- II.3) Using PYTHON `venv` . . . 12
  - II.3.a) Write a configuration file . . . 12
  - II.3.b) The environment creation in itself . . . 12

III) Installation guide in `README.md` . . . **13**
- III.1) For the virtual environment creation . . . 13
- III.2) Create an environment variable for the main repository . . . 14
  - III.2.a) Creation and use . . . 14
  - III.2.b) Update the installation guide . . . 16

TAF course (*Preliminary*)- October 8, 2026

I) Set up version control

I.1) Creation of a github repository

Here, we are going to follow this quickstart from the Github documentation.

The `README.md` file

The presence of such a file is very important. Ideally, there should be a `README.md` file in every directory within your repository.

> 💬 The `md` extension means "markdown". You can find many pieces of information about this online, but to get you started quickly, you could have a look at this cheat sheet.

I.2) Local creation of a git project

> 💬 There are several ways of doing it. Here we will create a clone of the Github repository we just created.

⚠ When pushing your first commit, push it in the main branch. Do not create a new branch yet, we will do it later.

Clone a Github repository

We will follow the procedure Cloning a repository available here.

> 💬 The cloning using SSH key would be the option to favour. However it requires SSH certificate authority : if you did not configure it, just go for the HTTPS option.

Ex. 1.1 | Create a local version of the remote repository

Clone your Github repository at the desired location on your machine.

Hint

You could for instance place it in your `HOME` :

```
$ cd  # go home
$ # follow git clone procedure
```

Listing 1.1 – In the terminal

I.3) Create a dev branch

Develop on a branch dedicated to development

It can seem trivial, but you should not develop your project on the `main` branch, even if you are the only contributor.
You should always create a branch dedicated to development, that you could call `dev` for instance. Every local modification of the project should be done on this branch. Then, you would push this branch on the remote Github repo to finally operate a `Pull Request` to merge the `main` and `dev` branches.

TAF course (*Preliminary*)- October 8, 2026

💬 This is unavoidable in a case of a multi-contributor project.
In the example of a group composed of Michael, Jim and Dwight, each one of them should create his own development branch :

```
dev_Michael
dev_Jim
dev_Dwight
```

Pull request

Pull requests are proposals to merge code changes into a project. A pull request is GitHub's foundational collaboration feature, letting you discuss and review changes before merging them. This helps teams work together, catch issues early, and maintain code quality. – Source

🚀 If you want to open a Pull request but that there is still some work to do or that you want the insight of colleagues before making it official, and good option is to create a Draft Pull request.

🚀 You can configure what are called Github Actions to automatise your Pull request and perform some checks before merging branches.

Create a dev branch (locally and remotely)

All the following instructions should be done in the terminal.

1. Go to the project repository
2. To create a branch locally, type

```
$ git checkout -b dev
```

💬 You can of course change `dev` for the name of your choice.

💬 The `checkout` command allows you to move to another branch, and the `-b` option allows you to create a new branch.

3. Get the name of the remote repo :

```
$ git remote -v
```

💬 We note it `REMOTE_REPO` here but you would need to change it by the actual result of the command above.

4. To also create this same branch remotely, push your newborn branch to `REMOTE_REPO` :

```
$ git push -u REMOTE_REPO dev
```

💬 The `-u` option in the `push` command is only needed for the first push.

TAF course (*Preliminary*)- October 8, 2026

Ex. 1.2 | Create a dev branch

Follow the method above to create your own branch dedicated to development, both locally and remotely.

At this point, you should have :

- ▷ a local Git repo with a `main` branch and a `dev` branch
- ▷ a remote Github repo with the same branches
- ▷ the `main` branch and a `dev` branch should be identical

⚠ From now on, and for the rest of this whole course, we will stay on the `dev` branch.

I.4) Open a Pull Request

Commit, push and open a Pull Request

Commit your modifications

1. Apply modifications to your project on the `dev` branch
2. Add your modifications :
   ```
   $ git add <put here the files you want to add>
   ```
3. Commit these changes :
   ```
   $ git commit -m "My commit for blablabla"
   ```

💬 This can be done numerous times without pushing.

Push your local version of the branch towards remote server

4. Push the new version of your branch to the remote repo :
   ```
   $ git push REMOTE_REPO dev
   ```

💬 This can be done numerous times without opening a Pull Request.

Open a Pull Request

5. Go to your Github repo, Github will automatically notice that the `dev` branch contains more commits than the `main` branch, and will propose you a Pull Request. You just have to follow the steps described on the web site.
   If you need more information, you can have a look at the documentation.

TAF course (Preliminary)- October 8, 2026

Ex. 1.3 | Make your first pull request

Update the `README.md` file by putting a title :

```
# Template of Analysis Framework (TAF)
```

Listing 1.2 – In `README.md`

Follow the procedure described above to :

1. make your first commit
2. push it
3. open your first Pull Request

💬 There are still many things that could be introduced concerning Git and Github. Here we shall stick to the basic tools presented above as we will not need more.
If you seek more content on this topic, there is a very nice Software Carpentry Workshop that should answer many of your interrogations.

II) Set up a virtual environment

II.1) Motivations

Use virtual environments

When we work in a "real" environment of the computer and we wish to install new PYTHON libraries, some conflicts between libraries may occur (notably regarding required versions).

Example

On our computer, the library A is installed with version 1.5. For some reason, we need to install a new library (the library B). However, the library B requires a version of library A inferior or equal to 1.3 : conflict!
At this point, one could believe that one "simply needs to re-install library A with version 1.3".

HOWEVER!

A new player enters the game : the library C, that has been installed on the computer all along. And, bad luck, this library C requires a version of library A superior or equal to 1.5.

Name of PYTHON library	Version	Necessity
A	1.5	/
B	/	$A \leqslant 1.3$
C	/	$A \geqslant 1.5$

TAF course (*Preliminary*)- October 8, 2026

We are stuck! We could delete the library C to use libraries A and B, but this would only be a temporary solution. Indeed, the day where we will need this library C once again, we will need to re-install it and remember this versions conflict issue between libraries A, B and C.
Hence this solution is not viable, especially since we usually have more than three libraries installed on our computer, and these are as many potential conflicts.

The previous example highlights the necessity (and practicality) to use separate virtual environments.

> 💬 Coming back to the previous example, one could create two virtual environments :
>
> ▷ an environment where libraries A and B live in harmony (without library C);
>
> ▷ an environment where libraries A and C live in harmony (without library B).

II.2) Using `conda`

II.2.a) Write a configuration file

> 💬 There are many ways of creating and configuring a virtual environment in `conda`. Here we choose the most direct, fast and modular method : a YAML configuration file.

YAML file

A YAML file is recognisable by its extension `.yml`. It is a file format very often used for configuration files and that must respect a well-defined structure.

Nota bene

We will use YAML files as configuration files for PYTHON scripts (see Section **??**).

YAML file to set up a virtual environment for TAF using `conda`
[Example]

```
name: taf_env
channels:
  - conda-forge
dependencies:
  - python=3.12
  - pyyaml
  - pandas
  - numpy
  - matplotlib
  - pip
  - pip:
    - argparse
```

Listing 1.3 – taf_env.yml

TAF course (Preliminary)- October 8, 2026

> 💬 The libraries under `dependencies` will be installed via `conda`. But if they are under `pip`, they will be installed via `pip`.
>
> ⚠ In this configuration file, the line
>
> ```
> - python=3.12
> ```
>
> allows to require the desired version of PYTHON.

II.2.b) The environment creation in itself

⚙ Create a virtual environment using `conda`

Once the preliminary work of creating the configuration file `taf_env.yml` is completed, one simply has to run a single command to create the virtual environment :

```
$ conda env create -f taf_env.yml
```

Listing 1.4 – In the terminal

⚙ Update an existing `conda` environment

If you wish to update your environment (*e.g.* add a library), you have to modify the YAML file and run :

```
$ conda env update --name taf_env --file taf_env.yml --prune
```

Listing 1.5 – In the terminal

ℹ Nota bene

One huge advantage of `conda`, that has no equivalent with PYTHON `venv` (see below), is the possibility to list all of the virtual environments created on the machine :

```
$ conda env list
```

Listing 1.6 – In the terminal

⚙ Enter/exit the environment

To enter the environment `taf_env` :

```
$ conda activate taf_env
```

Listing 1.7 – In the terminal

TAF course (*Preliminary*)- October 8, 2026

Nota bene

In your terminal, the information before the "$ " will indicate the name of the environment, like (it depends on the terminal) :

```
(taf_env) $
```

Listing 1.8 – In the terminal

To exit the environment :

```
(taf_env) $ conda deactivate
```

Listing 1.9 – In the terminal

II.3) Using PYTHON venv

💬 The creation is a bit less direct : one needs to first create the environment and then install the libraries once inside.

II.3.a) Write a configuration file

💬 venv does not use YAML files but simple ASCII `.txt` files.

Text file to set up a virtual environment for TAF using venv [Example]

```
pyyaml
pandas
numpy
matplotlib
argparse
```

Listing 1.10 – `requirements_py_venv_taf.yml`

💬 Note that the name of the environment is not specified here contrary to the `conda` method.

II.3.b) The environment creation in itself

Create a virtual environment using venv

To create a virtual environment named `taf_env` with version 3.12 (this is an example) of PYTHON, run :

```
$ python3.12 -m venv taf_env
```

Listing 1.11 – In the terminal

⚠ You have to choose the right location on your computer, because the files associated to this virtual environment will be created at this location.

TAF course (Preliminary)- October 8, 2026

Update an existing venv environment

1. Enter the environment :

```
$ source path/taf_env/bin/activate
```

Listing 1.12 – In the terminal

Of course, you will need to replace `path` by the location of your environment installation.

2. Run

```
$ pip install --upgrade pip
$ python3 -m pip install -r requirements_py_venv_taf.txt
```

Listing 1.13 – In the terminal

> 💬 Other tools exist (on top of conda and venv).

Ex. 1.4 | Create a virtual environment dedicated to TAF

Thanks to the methods described above, create a virtual environment dedicated to your Analysis Framework.

> 💬 You can use the tool you want between `conda` and `venv`.

TAF course (Preliminary) - October 8, 2026

III) Installation guide in `README.md`

Installation guide

It is a good habit to give some kind of installation guide in the `README.md` file.
This is useful for you, as it can help you keep track of the YAML file you used to create your virtual environment, but also to keep track of the evolution of this file along the project development (*e.g.* add libraries as the project gets bigger).

III.1) For the virtual environment creation

Here is a summary version of the explanations given in the sections above. You could just add it to your `README.md` file.

```
# Installation

## Create a virtual environment

### Option 1 -- using ‘conda‘

To create the virtual environment configured in the file ‘taf_env.yml‘, one simply needs
    to run:
‘‘‘
conda env create -f taf_env.yml
‘‘‘
```

````
*Note: to update an existing virtual environment (if one wished to modify ‘taf_env.yml‘),
     run*
‘‘‘
conda env update --name taf_env --file taf_env.yml --prune
‘‘‘

### Option 2 -- using python ‘venv‘

To create a virtual environment named ‘taf_env‘ with version 3.12 (this is an example) of
     Python, run:

‘‘‘
python3.12 -m venv taf_env
‘‘‘

*Careful: you have to choose the right location on your computer, because the files
    associated to this virtual environment will be created at this location.*

Once the environment is created, you can enter it via:

‘‘‘
source path/taf_env/bin/activate
‘‘‘

Once inside this environment, **and only once inside (!)**, run:
‘‘‘
pip install --upgrade pip
python3 -m pip install -r requirements_py_venv_taf.txt
‘‘‘

to install the requirements defined in the file ‘requirements_py_venv_taf.txt‘
````

Listing 1.14 – In `README.md`

Ex. 1.5 | Guide for the virtual environment

1. Copy-paste the markdown code given above into your own `README.md`
2. Commit this modification thanks to `git commit`

III.2) Create an environment variable for the main repository

III.2.a) Creation and use

Make the path to the main repository generic

When you are working locally on your machine, the path to the project repository (*i.e.* the so-called main repository) is not the same as the one on a colleague's machine. The solution is to create an environment variable for the main repository in the configuration file of the machine :

- ▷ `~/.bashrc` in Unix
- ▷ `~/.bash_profile` in MacOS

TAF course (*Preliminary*)- October 8, 2026

> 💬 In what follows, the configuration file will be referred as ~/.bash*
> It is up to you to choose the right name according to your operating system.

You should add the following line to this configuration file to create an environment variable named TAF :

```
export TAF="<put the path to your main repo>"
```

Listing 1.15 – In ~/.bash*

Make the new environment variable active

After adding the environment variable

```
export TAF="<put the path to your main repo>"
```

Listing 1.16 – In ~/.bash*

there are two methods to activate it :

- ▷ Method 1 : simply open a new terminal.
  > 💬 This automatically executes the ~/.bash* file.
- ▷ Method 2 : execute the ~/.bash* file manually :

  ```
  source ~/.bash*
  ```

  Listing 1.17 – In the terminal

  > 💬 This has the advantage of staying in the same terminal.

The main idea is that once you followed either one of these two methods, the variable TAF will be define (as long as you keep the dedicated line in the ~/.bash* file).

Use this variable

To access the value of the variable TAF, just type ${TAF}

Example

In the terminal, type

```
$ echo ${TAF}
```

to display the content of the variable TAF in the terminal, or

```
$ ls ${TAF}
```

to display the content of the main repository.

Ex. 1.6 | Create an environment variable for the main repository

Use the methods above (and their examples) to define a new environment variable called TAF, and make it active in your terminal.

TAF course (Preliminary)- October 8, 2026

⚠ From now on, the location of the main repository in the computer will be accessed via `${TAF}`

III.2.b) Update the installation guide

In the rest of the project, the path to the main repository will be replaced by the environment variable `TAF`, so it is absolutely crucial that every user defines this variable (otherwise, all the paths will be broken for the user!).
That's why you need to add pieces of information on how to define this variable and how to use it, inside the `README.md` in the `Installation` section :

```
## Create an environment variable for this repository

In your ‘~/.bashrc‘ (in Unix) or ‘~/.bash_profile‘ (in MacOS) add

‘‘‘
export TAF="<put the path to your local copy of this repo>"
‘‘‘

*Note: to get the local path to your repository, enter ‘pwd‘ in your terminal while being
    in your repo.*

*Note: the ‘export‘ keyword assumes that the shell is in BASH, it can vary if it is not (
   e.g. zsh, tcsh, ...).*

*Careful: do not forget to either open a new terminal or to ‘source‘ your ‘bash*‘ file to
    make this modification active.*

From now on, the location of this repo in the computer will be accessed via ‘${TAF}‘.
```

Ex. 1.7 | Guide for the environment variable

1. Copy-paste the markdown code given above into your own `README.md`
2. Commit this modification thanks to `git commit`

TAF course (*Preliminary*)- October 8, 2026

Chapter 2

PYTHON tips & tricks

Chapter outline

I) Set up Visual Studio Code for PYTHON **18**
II) Some PYTHON tips for aesthetics (but not only…) **19**
II.1) The `if __name__ == "__main__"` 19
II.2) The C-like architecture 22
II.3) Type hint 23
II.4) Comments & docstrings 24
II.4.a) Docstring for a function or a method 24
II.4.b) Docstring for a whole file 25
II.4.c) Highlighted comments 25
II.5) Add flags to the `main` function 26
II.6) LOG messages 27
II.6.a) The `Utils` folder 27
II.6.b) Create a logger module 27
II.6.c) Update `PYTHONPATH` 27
II.6.d) Update installation guide in `README.md` 29
II.7) Create a Visual Studio Code snippet 29
III) Some PYTHON tips for the actual development **31**
III.1) `f-string` 31
III.2) `list` 31
III.3) `dict` 33
III.4) Loop over iterables 34

A basic knowledge of PYTHON is required for this lecture. We will see features that you might already know and others that you do not (that will be of course of higher interest for you).

TAF course (*Preliminary*)- October 8, 2026

I) Set up Visual Studio Code for PYTHON

Open files or folders with Visual Studio Code

One option is to use the tools available in the `Explorer` of the Visual Studio Code window (top left).
Another option is to use the terminal :

```
$ code myFolder  # opens the whole folder named myFolder
# OR
$ code myFile  # opens the file named myFile
```

Use Visual Studio Code terminal

In the Visual Studio Code window, among the tabs on the top left, you have a tab entitled `Terminal`. Just click it to open a terminal within the Visual Studio Code window.

> ⚠ Sometimes Visual Studio Code terminal can be a bit capricious and less robust than the native OS terminal. Note that it should not be the case with the tools used in this course.

Useful extensions for PYTHON

In your Visual Studio Code window, click on the `Extensions` panel on the left tab. Then search and install the following extensions :

- `Python` : basic extension for PYTHON
- `Pylint` : to get hints on PYTHON writing conventions/guidelines
- `Black Formatter` : to auto format code
- `autoDocstring` : to automatically generate docstring (see Section Comments & docstrings)

Ex. 2.1 | Install extensions

Install the extensions mentioned above.

Tune your extensions with a GUI



1. Go to the `Preferences` menu.

   > 💬 On MacOS, it is in the `Code` tab on the top left. On Unix, it is in the `File` tab.

2. Click on `Settings`
3. In the pop-up window, look for the `Extensions` entry and choose the Extension you want to tune.

TAF course (*Preliminary*)- October 8, 2026

Tune your extensions via `settings.json`

The `settings.json` file is a configuration file. To open it, go to `View > Command Palette > Preferences: Open User Settings`

A window will pop up. It is a JSON file that can be written to set some settings.

Example

Here is an example of settings you could add to this file :

```
{
    "[python]": {
        "editor.defaultFormatter": "ms-python.black-formatter",
        "editor.formatOnSave": true,
    },
    "black-formatter.args": [
        "--line-length",
        "100"
    ],
    "autoDocstring.startOnNewLine": true,
    "files.autoSave": "afterDelay",
    "window.autoDetectColorScheme": true,
}
```

Listing 2.1 – `settings.json`

Ex. 2.2 | Tune `Black Formatter` and `autoDocstring`

1. Copy-paste the example above in your `settings.json` file. This will tune `Black Formatter` and `autoDocstring` extensions, and enable autosave.
2. Test if `Black Formatter` works fine. To do so, go in any PYTHON script, put extra spaces here and there, then save the document.
   It should autoformat itself.

💬 We will test `autoDocstring` in the next section.

II) Some PYTHON tips for aesthetics (but not only…)

II.1) The `if __name__ == "__main__"`

On the web, in many forums, documentations, examples, ... you might find this `if __name__ == "__main__"` condition. What does it do? What is it used for?
In PYTHON, there are basically two types of `.py` files :

- the ones that are meant to be executed (what we call *scripts*);
- the ones that are meant to be imported in other `.py` files, and not meant to be executed (what we call *modules*).

TAF course (Preliminary)- October 8, 2026

Nota bene

The *script* or *module* status of a file is context-dependent. Indeed, one can have a `.py` file that is meant to be a *module* but that could be executed for debugging purposes (for instance).

In order to distinguish between these two types, there is something called the `__name__` variable.

Ex. 2.3 | A repository for exercises

In `${TAF}` repo, create a directory `Exercise`

Hint

You can directly use the "`New Folder...` " functionality in Visual Studio Code, or do it via the terminal :

```
$ mkdir ${TAF}/Exercise
```

Listing 2.2 – In the terminal

Ex. 2.4 | The `__name__` variable

1. Enter the `Exercise` directory
2. Create a directory `TestNameVariable`
3. Enter this directory
4. Create a file `script.py` such as :

```
print("In script.py, __name__: ", __name__)
```

Listing 2.3 – script.py

5. Create a file `module.py` such as :

```
print("In module.py, __name__: ", __name__)
```

Listing 2.4 – module.py

6. Execute the codes using :

```
$ python3 script.py
$ python3 module.py
```

Listing 2.5 – In the terminal

What is printed in the terminal ?

7. Now modify `script.py` by importing `module.py` :

```
import module
print("In script.py, __name__: ", __name__)
```

Listing 2.6 – script.py

TAF course (*Preliminary*)- October 8, 2026

8 Execute `script.py` :

```
$ python3 script.py
```

Listing 2.7 – In the terminal

What is printed in the terminal ? Is there a difference with the previous message ? If so, why ?

The `if __name__ == "__main__"` condition

To ensure that a `.py` file will only be executed if it is the "main" one (*i.e.* the target of `python3` in the execution command line), one can add the

```
if __name__ == "__main__":
    # put code here
```

condition.

Ex. 2.5 | Add the `if __name__ == "__main__"` condition

Add the `if __name__ == "__main__"` condition to the files `module.py` and `script.py`

The `if __name__ == "__main__"` condition for a "main" script

In PYTHON, there is a convention : if your code is meant to be executed, you should embed it within the `if __name__ == "__main__"` condition.

> 💬 As a consequence, all the codes in scripts lie inside the
>
> ```
> if __name__ == "__main__":
>     # put code here
> ```
>
> block.
>
> 💬 We will see a "cleaner" way to proceed below.

Example

Let's modify `script.py` from the previous exercise as follows :

```
import module
if __name__ == "__main__":
    print("Hello from script.py")
```

Listing 2.8 – script.py

The execution gives this result :

```
$ python3 script.py
In module.py, __name__:  module
Hello from script.py
```

Listing 2.9 – In the terminal

TAF course (*Preliminary*)- October 8, 2026

The code from `script.py` is executed as it is the main code. However, the code of `module.py` is also executed while it has the status of *module*...

The `if __name__ == "__main__"` condition for debugging/testing a module

An exception to the previous convention can happen in the following case : you have a module and want to perform direct debugging or testing. Then, you put this code inside a `if __name__ == "__main__"` condition so it does not get executed when the module is imported.

Example

Coming back to the example in the box above. If we want the code of `module.py` to be executed only if it is directly executed and not if it is imported, we should do :

```
if __name__ == "__main__":
    print("Hello from module.py")
    print("In module.py, __name__: ", __name__)
```

Listing 2.10 – module.py

When we want to debug/test the module, we can do :

```
$ python3 module.py
Hello from module.py
In module.py, __name__:  __main__
```

Listing 2.11 – In the terminal

and the code in `module.py` is executed.
However, if we want to use it as a *module* :

```
$ python3 script.py
Hello from script.py
```

Listing 2.12 – In the terminal

💬 Note that the code of `module.py` was not executed in this last command line.

II.2) The C-like architecture

The `main` function

We just saw that a PYTHON script should possess the `if __name__ == "__main__"` condition, and that its code should lie inside the condition block. This is not optimal, as it can alter the code readability.

To overcome this issue, we adopt an "architecture" inspired by the C language : a `main` function.

```
def main():
    print("This is my main function")
```

TAF course (Preliminary)- October 8, 2026

```python
if __name__ == "__main__":
    main()
```

Then, the "main" code will sit inside the `main` function and the `if __name__ == "__main__"` condition only embeds the call of the `main` function.

Why is it a good way to go ?

- ▷ in one quick look at the very end of the script, on can easily get a peak at the structure of the script ;
- ▷ if one wants a better understanding of the content of the script, one sees that there is a `main` function that is called, goes check what is inside this `main` function, that may itself call some helper functions, etc. . .

Ex. 2.6 | `main` function convention

In `script.py`, implement the `main` function convention.

II.3) Type hint

Type hint

In PYTHON, type hints allow developers to *annotate* their code with *expected types* for variables and function arguments and return. A type hint is written as follows :

```
variable: type = value
```

> 💬 This can make the code more easily readable and also, possibly, allow to catch errors before runtime. The latter requires the use of external tools : so-called *type checkers*.

> 💬 This looks a lot like C for instance, except that in C, if you do not specify the type of your variable, the code will not compile, while in PYTHON it is just a hint : it will not lead to errors (if you do not apply some external type checker).

Example

```python
a: int = 1
pi: float = 3.14
name: str = "Jim"
names: list[str] = ["Michael", "Jim", "Dwight"]

def print_hello() -> None:
    print("Hello")

def sum(x: float, y: float) -> float:
    return x + y
```

TAF course (Preliminary)- October 8, 2026

Ex. 2.7 | Type hinting

In `script.py`, add a type hint to the `main` function.

II.4) Comments & docstrings

In your VS Code window, you should see that `def main` is underlined (thanks to Pylint extension). If you hover your mouse over `def main`, a window will pop up with a message saying "Missing function or method docstring".

docstring

It is an element of code that will not be interpreted (it has the status of a comment) and that aims at providing pieces of information regarding a given function or the whole document.
The docstring environment is surrounded by triplets of " :

```
"""
Docstring here
"""
```

docstring

Once written, the docstring can be used to generate a documentation.

> This aspect will not be addressed in this course.

Also, in VS Code (for instance), if you hover the mouse over some function (and if its docstring is written), this gives you direct access to its documentation via a pop-up window.

II.4.a) Docstring for a function or a method

Use `autoDocstring`

Type `"""` right after the `:` of a function definition. An auto-completion pop-up will appear : press `Tab` or `Enter` to run `autoDocstring`.

Ex. 2.8 | Docstring the `main` function

Use `autoDocstring` on the `main` function of `script.py`

Ex. 2.9 | Docstring a function with arguments and a return

In `script.py`, define a dummy function like :

```
def sum(a: float, b: float) -> float:
    return a + b
```

Use `autoDocstring` on the `sum` function.

TAF course (*Preliminary*)- October 8, 2026

II.4.b) Docstring for a whole file

Docstring of a file

A `.py` file should start with a docstring like the following one :

```
"""
file: myscript.py
brief: Simple script to show how to define dosctrings
usage: python3 myscript.py
note:
author: Alexandre BIGOT, alexandre.bigot@iphc.cnrs.fr
"""
```

> 💬 This is an example, there are several ways to format such a "file docstring".

Ex. 2.10 | Docstring a file

Docstring the file `script.py`

II.4.c) Highlighted comments

Highlighted comments

Comments containing a specific keyword that allows for them to be highlighted in your text editor (Visual Studio Code here).

Nota bene

The keywords used most often are `TODO` and `FIXME`.

Example

Let's suppose you are developing a code and do not have time to implement a given functionality at this very instant. In order to keep track of this in your code, you could put a comment like this one :

```
# TODO: add functionality blablabla
```

Also, if you are implementing a temporary solution or that you think a part of your code requires some debugging, you could add a comment like this one :

```
# FIXME: temporary solution
```

Ex. 2.11 | Highlighted comment

In `script.py`, add a `TODO` comment indicating that you will need to add a debug flag (see next section).

TAF course (Preliminary)- October 8, 2026

Nota bene

The docstring (and comments in general actually) might seem anecdotal at first sight (especially when you are developing your code, on the moment you just want to move forward and not lose time documenting the code...) but it is very important for one simple reason : you should not forget that you are developing code for your future self and/or other users !

- Your future self might be you in
  - 1 week : in this case, you should still have a good and direct understanding of the code because it is still fresh in your memory ;
  - 1 month : you could have forgotten some details here and there but you could still catch up quite fast ;
  - or 2 years : then it can become quite difficult to understand the code and what you had in mind at the time you wrote it.
- For other users, it is even harder because they are not in your head and it can be time-consuming to explain them the code (which would require you to master the code, looping back to the previous item).

To wrap up :

At the moment you write the code, you should consume a little bit of time to write a clear documentation in order to save lots of time in the future ☺

II.5) Add flags to the `main` function

Flag

A flag is a boolean variable, a switch that may be on (`True`) or off (`False`).

Flag in the `main` function

It is a flag that is an argument of the `main` function.

DEBUG flag

It can be very useful (it is recommend to do it every time) to define a **DEBUG** flag, as follows :

```
def main(debug: bool) -> None:
    """
    Main function

    Parameters
    ----------------------------------------------
    - debug: bool
        Switch for debugging
    """
    if debug:
        print("DEBUG mode activated!")
```

TAF course (*Preliminary*)- October 8, 2026

```
if __name__ == "__main__":
    DEBUG: bool = True
    main(DEBUG)
```

This way, you can put some debugging messages or checks in your `main` function via `if debug:` environments.

> 💬 Note that the `DEBUG` variable given as argument to the call of the `main` function is in capital letters as it is a constant (this is a PYTHON convention).

Ex. 2.12 | `DEBUG` flag

In `script.py`, add a `DEBUG` flag.

> 💬 Once this is done, you can remove the `TODO` comment added in Ex. 2.11.

II.6) LOG messages

II.6.a) The `Utils` folder

`Utils` folder

Generic name for a folder that shall welcome all relevant modules (for data handling, analysis purposes, plots production, ...) throughout the project.

Ex. 2.13 | Creation of the `Utils` folder

At the top level of your `TAF` project (*i.e.* in the `${TAF}` repo), create a folder named `Utils`

II.6.b) Create a logger module

Ex. 2.14 | Logger module

Inside the `Utils` folder, download the PYTHON module named `taf_logger.py` available in the Seafile folder.

II.6.c) Update `PYTHONPATH`

Now, if we want to import the `Logger` class from the `taf_logger.py` module, we must give the position of this module in our machine (so PYTHON can know its location and import it). To avoid this issue, one can modify the `PYTHONPATH` environment variable.

> 💬 This is *a* possible solution. There are others that can be quite complex.

TAF course (Preliminary)- October 8, 2026

Add `Utils` folder to the path of known PYTHON modules

Add the following line in the ~/.bashrc (in Unix) or ~/.bash_profile (in MacOS) :

```
export PYTHONPATH="${PYTHONPATH}:${TAF}/Utils"
```

⚠ Do not forget to source your ~/.bash* file or to open a new terminal to make this modification active.

💬 Remember we already mentioned environment variables in Section Create an environment variable for the main repository.

Ex. 2.15 |

Add `Utils` folder to the path of known PYTHON modules.

Ex. 2.16 | Test if this updated PYTHONPATH works

1. Enter the `${TAF}/Exercise` directory
2. Create a `Logger` directory and enter it
3. Create a file `test_logger.py`, and paste the following code in it :

```python
"""
file: test_logger.py
brief:
usage: python3 test_logger.py
note:
author: Alexandre BIGOT, alexandre.bigot@iphc.cnrs.fr
"""

from taf_logger import Logger as LOG


def main(debug: bool) -> None:
    """
    Main function

    Args:
        debug (bool): Switch for debugging
    """
    if debug:
        LOG("Debug mode activated!", "DEBUG")


if __name__ == "__main__":
    DEBUG: bool = True
    main(DEBUG)
```

4. Execute the script and check if it works fine.

TAF course (Preliminary)- October 8, 2026

Ex. 2.17 | LOG messages

We will add some content to `test_logger.py` :

1. a `RESULT` message
2. a `INFO` message
3. a `WARNING` message
4. a `ERROR` message
5. a `FATAL` message

What happens when you add a `FATAL` message ?

II.6.d) Update installation guide in `README.md`

⚠ The previous modification on `PYTHONPATH` redefines the known paths for PYTHON `import` command. If a third-party user is not aware of this, the scripts will not work on his/her machine!
It is mandatory to indicate it in `README.md`

```
## Python path to 'Utils' modules

In order to be able to easily import TAF 'Utils' modules in local scripts, a solution is
    to add the following line to your '~/.bashrc' or '~/.bash_profile':
'''
export PYTHONPATH="${PYTHONPATH}:${TAF}/Utils"
'''

*Note: doing so, we will not need to specify the path of the module relative to the
    script we are developping.*

*Careful: do not forget to either open a new terminal or to 'source' your 'bash*' file to
     make this modification active.*
```

Listing 2.13 – In `README.md`

Ex. 2.18 | Guide for the `PYTHONPATH` update

1. Copy-paste the markdown code given above into your own `README.md`
2. Commit this modification thanks to `git commit`

II.7) Create a Visual Studio Code snippet

Add a Visual Studio Code snippet

1. Open a Visual Studio Code window
2. Select `Configure Snippets` in `File > Preferences` (for Unix) or in `Code > Preferences` (for MacOS)
3. Select the language (PYTHON)

TAF course (*Preliminary*)- October 8, 2026

> 💬 This opens the file `python.json` containing the definitions of snippets.

Ex. 2.19 | Add a snippet for PYTHON template and use it

1. Add the following lines between the main `{ }` of the `python.json` file.

```
"TAF Python File Template": {
    "prefix": "py_tmpl_taf",
    "body": [
        "\"\"\"",
        "file: ${TM_FILENAME}",
        "brief:",
        "usage: python3 ${TM_FILENAME}",
        "note:",
        "author: Name, email address",
        "\"\"\"",
        "",
        "from taf_logger import Logger as LOG",
        "",
        "",
        "def main(debug: bool) -> None:",
        "    \"\"\"",
        "    Main function",
        "",
        "    Args:",
        "        debug (bool): Switch for debugging",
        "    \"\"\"",
        "    if debug:",
        "        LOG(\"Debug mode activated!\", \"DEBUG\")",
        "",
        "",
        "if __name__ == \"__main__\":",
        "    DEBUG: bool = True",
        "    main(DEBUG)",
        ""
    ],
    "description": "Template for new Python files in TAF"
}
```

Listing 2.14 – To put between the global `{ }` envrionment

2. Create a new `.py` file with any name and open it in Visual Studio Code
3. Type `py_tmpl_taf`

> 💬 You should see some auto-completion proposition : press `Tab` or `Enter` to accept it and run the snippet.

Ex. 2.20 | Commit and push the content of this Section

1. `git add` the `Exercise` and `Utils` folders

> 💬 No need to `add` the `README.md` as it was already added (and commited) in Ex. 2.18.

TAF course (Preliminary)- October 8, 2026

2. `git commit` these modifications
3. Push your `dev` branch

III) Some PYTHON tips for the actual development

Ex. 2.21 | Sandbox

1. Enter the `${TAF}/Exercise` directory
2. Create a `PythonTips` directory
3. Among the tools introduced in this section, take some time to perform some tests (*e.g.* copy-paste the codes given as examples in dedicated scripts) whenever you encounter a new notion that you are not sure to fully understand.

III.1) `f-string`

`f-string`

`f-strings` are string literals prefixed with `"f"` that contain expressions inside curly braces {}.

Example

```
pi = 3.141592
my_fstring = f"pi = {pi}"
print(my_fstring)
my_string_with_4decimals = f"pi = {pi:.4f}"
```

Listing 2.15 – `example.py`

```
$ python3 example.py
pi = 3.141592
pi = 3.1415
```

III.2) `list`

PYTHON lists are one of the most important features you need to master.

Construct a `list` object

Use the `[]` notation. The comma `", "` is used to separate the elements within a list.

```
$ python3
>>> lempty = []
>>> print(lempty)
[]
```

TAF course (Preliminary)- October 8, 2026

```
>>> l1 = [1, 2, 3, 4]
>>> l2 = ["d", "w", "i", "g", "h", "t"]
>>> l3 = [None, 3.14, None]
>>> l1 + l2 +l3
[1, 2, 3, 4, 'd', 'w', 'i', 'g', 'h', 't', None, 3.14, None]
```

💬 The + operator is used to concatenate lists.

Nota bene

A PYTHON list can accept multiple types of variables.

⚠ `list` is the name of the constructor of the `list` class, so you cannot name a variable `list`!

Create a list of size n

It can sometimes be better suited to work with a list of size `n` than with an empty list.

```
$ python3
>>> n = 5
>>> [False] * n
[False, False, False, False, False]
```

⚠ The list is filled with the same value for each entry.

Use a PYTHON expression to create a list

Let's suppose you want to create a list with integers from 0 to 9. You could do

```
$ python3
>>> l = []
>>> for i in range(10):
...     l.append(i)
...
>>> print(l)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

But there is a one-liner way thanks to a PYTHON expression :

```
$ python3
>>> l = [i for i in range(10)]
>>> print(l)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

💬 There is also a way to do this thanks to `numpy` :

```
$ python3
>>> import numpy as np
>>> l = np.arange(0, 10)
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

But then `l` would be a numpy array and not a list.

TAF course (*Preliminary*)- October 8, 2026

III.3) `dict`

⚠ A basic understanding of dictionaries is mandatory for the part of the lecture dedicated to the use of configuration files (see Section **??**).

Dictionary (`dict` class)

Dictionary is a data structure that stores information in key-value pairs.

> 💬 It resembles the `map` in C++.

> 💬 In PYTHON, a dictionary is an object of type `dict`

To build a dictionary, you can use the `{ ... }` notation :

```
my_dict = {"key1": value1, "key2": value2}
```

⚠ `dict` is the name of the constructor of the `dict` class, so you cannot name a variable `dict`!

⚠ The *keys* must be unique and immutable (like strings or numbers).
The *values* may be of any data type, whether mutable or immutable.

Nota bene

This implies that a *value* can be a `list` or even a `dict`! It is not necessarily a `int`, `float` or a `string`.

> 💬 Dictionaries are especially suited for accessing data by a specific name rather than a numeric position like in `list`.

Manipulate a `dict` object

We use the following example :

```
>>> salesman = {"name": "Dwight", "age": 30}
>>> salesman
{'name': 'Dwight', 'age': 30}
```

To retrieve the value of a given *key*, you need to put the key between `[ ]` after the dictionary name. For instance :

```
>>> salesman["name"]
'Dwight'
```

To add a new key-value pair, this is the exact same notation, except that you add a `= value`. For instance :

```
>>> salesman["rank"] = "Assistant (to the) regional manager"
>>> salesman["rank"]
'Assistant (to the) regional manager'
>>> salesman
{'name': 'Dwight', 'age': 30, 'rank': 'Assistant (to the) regional manager'}
```

You can also set a new value for an existing key :

```
>>> salesman["rank"] = "Regional manager"
>>> salesman["rank"]
```

TAF course (Preliminary)- October 8, 2026

```
'Regional manager'
>>> salesman
{'name': 'Dwight', 'age': 30, 'rank': 'Regional manager'}
```

The `items` method

To extract the key-value pairs, you can use :

```
key_value_pairs_items = my_dict.items()
```

Nota bene

This is a must when looping over a dictionary (see next section).

If you try to print it right as it is, you will not be able to manipulate it as a `list` :

```
>>> salesman.items()
dict_items([('name', 'Dwight'), ('age', 30), ('rank', 'Regional manager')])
>>> list(salesman.items())
[('name', 'Dwight'), ('age', 30), ('rank', 'Regional manager')]
```

III.4) Loop over iterables

Iterable

An iterable corresponds to any PYTHON object capable of returning its members one at a time. As a consequence, it to be iterated over in a for-loop.

In the methods below, we will consider a list :

```
names = ["Michael", "Jim", "Dwight"]
```

Loop over indices only

We use the `range` method :

```
for iname in range(len(names)):
    print(f"Index: {iname}, element: {names[i]}")
```

- This requires to access the length of the list, using `len`
- This method gives access to both elements (indirectly) and indices.

Nota bene

The iteration index is here called `iname` and not `i` for instance. This may seem anecdotal but this is quite important : remember you should give meaning to your variables names!

TAF course (*Preliminary*)- October 8, 2026

Loop over elements only

```
for name in names:
    print(f"Element: {name}")
```

> 💬 This is what we call a "for each" loop : for each iteration, we access the element of the list.

> 💬 This only gives access to the elements and not to their index in the list.

Loop on both indices and elements
The `enumerate` keyword

```
for iname, name in enumerate(names):
    print(f"Index: {iname}, element: {name}")
```

> 💬 This method gives direct access to both elements and indices.

`enumerate` function

Function used to iterate over an *iterable*[a], returning pairs in the form of tuples shaped as `(index, element)`.

Example

```
>>> names = ["Michael", "Jim", "Dwight"]
>>> print(f"Print enumerate output: {enumerate(names)}")
Print enumerate output: <enumerate object at 0x107943290>
>>> print(f"Print list of enumerate output: {list(enumerate(names))}")
Print list of enumerate output: [(0, 'Michael'), (1, 'Jim'), (2, 'Dwight')]
```

Listing 2.16 – In PYTHON console

> 🚀 One needs to convert enumerate output into a list otherwise we do not see what is within via print

a. It can be a `list` object or a `dict_items` object, for instance.

Loop over a dictionary

As mentioned in the previous section, a good option is to use the `items` method.

Example

Let's come back to the example of the previous section :

```
>>> salesman
{'name': 'Dwight', 'age': 30, 'rank': 'Regional manager'}
>>> for key, value in salesman.items():
...     print(f"key: {key}, value: {value}")
```

TAF course (Preliminary)- October 8, 2026

```
...
key: name, value: Dwight
key: age, value: 30
key: rank, value: Regional manager
```

`zip` function

Used to combine multiple iterable objects (*e.g.* lists, tuples, or strings) into a single iterable of tuples.

Example

```
>>> names_men = ["Michael", "Jim", "Dwight"]
>>> names_women = ["Holly", "Pam", "Angela"]
>>> for name_man, name_woman in zip(names_men, names_women):
...     print(f"{name_man} and {name_woman} form a couple")
...
Michael and Holly form a couple
Jim and Pam form a couple
Dwight and Angela form a couple
```

Listing 2.17 – In PYTHON console

⚠ This example shows a `zip` on two iterables, but we could also do it on three or more iterables.

`enumerate` and `zip` combo

As the output of `zip` is an iterable, one can use `enumerate` : to make it short, one can `enumerate` a `zip`!

Example

```
>>> names_men = ["Michael", "Jim", "Dwight"]
>>> names_women = ["Holly", "Pam", "Angela"]
>>> for icouple, (name_man, name_woman) in enumerate(zip(names_men, names_women)):
...     print(f"Couple {icouple}: {name_man} and {name_woman}")
...
Couple 0: Michael and Holly
Couple 1: Jim and Pam
Couple 2: Dwight and Angela
```

Listing 2.18 – In PYTHON console

TAF course (*Preliminary*)- October 8, 2026